Дорожная карта проекта: Собственный DNS-резолвер с внешним кэшем и обходом

---

1. Анализ ТЗ и распределение ролей

Технический профиль проекта

Проект является **исследовательско-лабораторным** (не production-разработка), где ключевая ценность — **воспроизводимый протокол действий со скриншотами**. Это смещает акцент с разработки кода на системное администрирование, сетевую инженерию и документирование.

Распределение ролей

#	Роль	Зона ответственности	Ключевые компетенции
P1	**DNS/Infrastructure Engineer** (Lead)	Установка и конфигурация DNS-резолвера (Unbound/PowerDNS Recursor), настройка зон DNSSEC, трассировки	BIND/Unbound, dig, DNS-протокол, Linux
P2	**Backend/Cache Engineer**	Установка и интеграция Redis, написание скриптов-прокси (Python/Lua), управление TTL в Redis	Redis, Python/Lua, REST API, Linux
P3	**DevOps/Lab Engineer**	Развёртывание стенда (VM/Docker), сетевая конфигурация, сборка кастомных конфигов, автоматизация	Docker, Vagrant, iptables, Wireshark
P4	**QA/Technical Writer**	Тестирование каждого сценария, сбор скриншотов, написание протокола действий, верификация баллов по ТЗ	dig, nslookup, Wireshark, Markdown/LaTeX

2. Декомпозиция проекта (High-Level)

Главная техническая цель

Развернуть DNS-резолвер (Unbound) с интеграцией Redis-кэша, обеспечить управление TTL независимо от стандарта, и продемонстрировать локальную подмену DNSSEC-зоны с документированием каждого шага.

Выбор ПО (обоснование до старта)

- **DNS-резолвер:** `Unbound` — поддерживает prefetch, Redis-модуль (`unbound-redis`/`redis-cache`), local-zone, богатая документация
- **Внешний кэш:** `Redis` — нативная поддержка TTL на уровне ключей, Lua-скрипты
- **Стенд:** Ubuntu 22.04 LTS (VM или Docker Compose)
- **Инструменты проверки:** `dig`, `drill`, `wireshark`, `redis-cli`

Ключевые Milestone

```
Week 1: [FOUNDATION]   Стенд готов + Резолвер установлен + Базовое
кэширование работает
Week 2: [CACHING]      Стандартный кэш исследован по всем 4 подпунктам
(1.1.А-Г)
Week 3: [REDIS]        Внешний Redis-кэш подключён и верифицирован (1.2.А-Г)
Week 4: [DNSSEC]       Локальная зона DNSSEC подменена + протокол завершён
(1.3.А-Г)
```

---

3. Еженедельный план производства (Спринты)

Обозначения: ● Критический путь | 📸 Требует скриншота | ✅ Критерий приёмки

Неделя (Спринт)	Milestone (Цель недели)	Конкретные задачи	Ответственная роль	Артефакт (Результат недели)
Неделя 1 Sprint 1	**FOUNDATION** Стенд развёрнут, резолвер запущен, базовая резолюция работает	**[P3]** 1. Создать lab-окружение: VM Ubuntu 22.04 или Docker Compose с сетью `dns-lab` (подсеть 10.10.0.0/24)	P3	✅ `docker-compose.yml` / Vagrantfile в репо
		[P3] 2. Установить Unbound: `apt install unbound`, проверить версию, убедиться в наличии модульной поддержки (`unbound -V | grep modules`) 📸	P3	✅ Unbound запущен, `systemctl status unbound` — active
		[P1] 3. Базовая конфигурация `unbound.conf`: `interface: 0.0.0.0`, `access-control: 10.10.0.0/24 allow`, `verbosity: 2`, отключить `do-not-query-localhost` 📸	P1	✅ Конфиг-файл `unbound.conf` v1.0
		[P1] 4. Установить корневые подсказки (`root.hints`), настроить DNSSEC-валидацию (`auto-trust-`	P1	✅ Успешная резолюция yandex.ru с AD-флагом

Неделя (Спринт)	Milestone (Цель недели)	Конкретные задачи	Ответственная роль	Артефакт (Результат недели)
		`anchor-file`), проверить `dig @localhost yandex.ru` 📸		
		[P4] 5. Написать **тест-план** для недель 2-4 (чеклист из 16 подпунктов ТЗ), создать структуру папок для скриншотов (`/docs/screenshots/week{1-4}/`)	P4	✅ `TEST_PLAN.md` с критериями прохождения каждого пункта
		[P2] 6. Установить Redis: `apt install redis-server`, проверить `redis-cli ping`, ознакомиться с модулем `unbound-redis` (или альтернативой `unbound-mod-redis`) 📸	P2	✅ Redis запущен, задокументированы версии ПО
		[ALL] 7. **Sprint Review:** демонстрация рабочего стенда, `dig @10.10.0.2 yandex.ru A` возвращает IP, команда согласует архитектуру интеграции Redis	ALL	✅ Протокол Week 1 (черновик)
Неделя 2 Sprint 2	**CACHING** Исследование стандартного кэша Unbound по подпунктам 1.1.А-Г	**[P1] [1.1.А]** Запросить `yandex.ru A`, зафиксировать TTL в ответе (`dig +ttl`), повторить запрос через N секунд и убедиться, что TTL убывает (кэш работает). 📸 Скриншот двух последовательных `dig`-запросов с разным TTL	P1 + P4	✅ **5 баллов** — доказательство убывания TTL
		[P1] [1.1.Б] Включить prefetch в `unbound.conf`: `prefetch: yes`, `prefetch-key: yes`. Выполнить нагрузочный тест (`dnsperf` или bash-loop), дождаться момента когда TTL < 10% — убедиться, что Unbound обновляет запись фоново, не возвращая просроченный ответ. 📸 Лог Unbound с записью `prefetch`	P1 + P4	✅ **5 баллов** — prefetch активен и логируется
		[P1] [1.1.В] Установить принудительный	P1 + P4	✅ **5 баллов** — TTL принудительно изменён

Неделя (Спринт)	Milestone (Цель недели)	Конкретные задачи	Ответственная роль	Артефакт (Результат недели)
		минимальный TTL: `cache-min-ttl: 300` и максимальный `cache-max-ttl: 86400`. Запросить домен с коротким TTL (например, `api.telegram.org`), убедиться, что резолвер отдаёт TTL не менее 300 сек. 📷 `dig` + значение TTL в ответе		
		[P1+P2] [1.1.Г] ● Исследование лимита кэша: установить `msg-cache-size: 1m` (минимум), `rrset-cache-size: 512k`. Заполнить кэш массовыми запросами (скрипт из 10 000 уникальных поддоменов через `dnsmasq` mock или публичные домены). Зафиксировать в логах факт eviction (`cache evicted`) — даже при `cache-min-ttl` запись вытесняется. 📷 Лог с eviction + `unbound-control stats` до и после	P1 + P2 + P4	✅ **10 баллов** — факт принудительной очистки задокументирован
		[P3] Настроить `unbound-control` (TLS-ключи), протестировать `unbound-control dump_cache` и `unbound-control load_cache` для дампа состояния	P3	✅ `unbound-control` работает, дамп кэша получен
		[P4] Оформить протокол Week 2 со скриншотами для пунктов 1.1.А-Г, верифицировать соответствие критериям баллов	P4	✅ `PROTOCOL_WEEK2.md` + папка `screenshots/week2/`
		[ALL] Sprint Review: демонстрация всех 4 сценариев 1.1, code review конфигов Unbound	ALL	✅ Unbound конфиг v2.0, протокол Week 2 (финал)
Неделя 3 Sprint 3	**REDIS CACHE** Подключение внешнего	**[P2] [1.2.А]** ● Подключить Redis к Unbound. Варианты: (a) `unbound-redis` module (если доступен в сборке), (b)	P2 + P3	✅ **10 баллов** — Redis содержит DNS-записи

Неделя (Спринт)	Milestone (Цель недели)	Конкретные задачи	Ответственная роль	Артефакт (Результат недели)
	Redis-кэша и управление TTL на уровне БД (пп. 1.2.А-Г)	написать Python/Lua-прокси, работающий как `dns-proxy`: перехватывает ответ Unbound, пишет в Redis с кастомным TTL, при повторном запросе отвечает из Redis до истечения TTL. Установить, настроить, проверить `redis-cli monitor`. 📷 `redis-cli keys *` с DNS-записями		
		[P2] [1.2.Б] Проверить механизм кэширования: сделать запрос → убедиться что запись попала в Redis (`HGET dns:yandex.ru A`), сделать повторный запрос → убедиться что ответ пришёл из Redis (TTL в Redis не совпадает с DNS TTL). 📷 `redis-cli` до и после + время ответа	P2 + P4	✅ **5 баллов** — кэширование через Redis подтверждено
		[P2] [1.2.В] ● Реализовать механизм ответов **из Redis**: DNS-прокси при наличии актуальной записи в Redis НЕ идёт к Unbound/upstream, а возвращает ответ напрямую. Трассировка: `dig +trace` показывает остановку на уровне локального резолвера. 📷 Wireshark — пакеты не уходят на upstream	P2 + P1 + P4	✅ **10 баллов** — ответ из Redis без upstream-запроса
		[P2] [1.2.Г] Настроить кастомное время хранения: записать `yandex.ru` с TTL=3600 секунд (игнорируя реальный DNS TTL=300). Проверить через `redis-cli TTL dns:yandex.ru` — значение ~3600. Дождаться истечения, убедиться что запись удалена. 📷 `redis-cli TTL` в момент записи и за 10 сек до истечения	P2 + P4	✅ **10 баллов** — кастомный TTL в Redis работает

Неделя (Спринт)	Milestone (Цель недели)	Конкретные задачи	Ответственная роль	Артефакт (Результат недели)
		[P3] Оформить `docker-compose.yml` с сервисами: `unbound`, `redis`, `dns-proxy` (Python/Lua), добавить `healthcheck`	P3	✅ `docker-compose.yml` v2.0 — стенд поднимается одной командой
		[P4] Оформить протокол Week 3 (пп. 1.2.А-Г), интегрировать в общий документ	P4	✅ `PROTOCOL_WEEK3.md` + `screenshots/week3/`
		[ALL] Sprint Review: интеграционный тест полного стека Unbound+Redis+Proxy, code review Python/Lua кода	ALL	✅ Redis-интеграция работает end-to-end
Неделя 4 Sprint 4	**DNSSEC BYPASS** Локальная подмена DNSSEC-зоны + финальный протокол (пп. 1.3.А-Г)	**[P1] [1.3.А]** Выбрать зону с DNSSEC (рекомендуется: `nic.ru` — имеет DS-запись, или собственный домен с включённым DNSSEC). Разместить локальную зону в Unbound: `local-zone: "nic.ru" static` с подменными A-записями. 📸 `dig nic.ru DS @8.8.8.8` (DNSSEC активен) vs `dig nic.ru A @localhost` (локальный ответ)	P1 + P4	✅ **5 баллов** — зона размещена локально
		[P1] [1.3.Б] Настроить Unbound: `local-zone: "nic.ru" redirect` + `local-data: "nic.ru A 192.168.1.100"`. Проверить, что при ВКЛЮЧЁННОЙ DNSSEC-валидации Unbound отвечает из локальной зоны (обходит BOGUS-статус). Важно: `domain-insecure: "nic.ru"` позволяет отключить валидацию для конкретного домена. 📸 `dig nic.ru A @localhost` — ответ из локальной зоны	P1 + P4	✅ **5 баллов** — локальная зона имеет приоритет над upstream
		[P1+P2] [1.3.В] Собрать реальные данные зоны: `dig nic.ru ANY @ns1.nic.ru` (все типы записей), `dig nic.ru DNSKEY`, `dig nic.ru RRSIG`. Закэшировать через	P1 + P2 + P4	✅ **10 баллов** — zone-файл создан из реальных данных

Неделя (Спринт)	Milestone (Цель недели)	Конкретные задачи	Ответственная роль	Артефакт (Результат недели)
		Redis (скрипт сбора). Создать zone-файл (`nic.ru.zone`) для Unbound в формате RFC 1035. 📸 Zone-файл + `unbound-checkconf` без ошибок		
		[P1] [1.3.Г] Финальная демонстрация: `dig +trace nic.ru A @localhost` — трассировка показывает, что рекурсия **останавливается на локальном резолвере**, upstream-серверы не опрашиваются. Wireshark: нет UDP/TCP пакетов к NS-серверам nic.ru. 📸 Wireshark capture + `dig +trace` вывод	P1 + P4	✅ **5 баллов** — трассировка подтверждает bypass
		[P4] Собрать итоговый протокол: объединить Week 1-4, оформить `README.md` с оглавлением, пронумеровать все скриншоты, добавить раздел "Выводы" (чем опасен bypass DNSSEC)	P4	✅ Итоговый документ `FINAL_PROTOCOL.pdf/md`
		[P3] Финальный smoke-test всего стенда с нуля (`docker-compose down -v && docker-compose up`), убедиться в воспроизводимости	P3	✅ `docker-compose.yml` v3.0 (финал)
		[ALL] Sprint Review / Demo Day: полная демонстрация по всем 16 пунктам ТЗ, подсчёт баллов (макс. 100+15=115)	ALL	✅ Финальный протокол + работающий стенд

---

4. Критический путь и технические риски

Визуализация критического пути

```
[Стенд готов] ──→ [Unbound конфиг] ──→ [1.1.A-B] ──→ [1.1.Г eviction]
                                                          │
                                                          ▼
                                                  [Redis интеграция 1.2.A] ●
                                                          │
                                                  [DNS из Redis 1.2.B] ●
                                                          │
                                                  [DNSSEC bypass 1.3.Б+Г] ●
```

● Риск 1: Интеграция Redis с Unbound (Bottleneck Week 3)

Проблема: Нативный модуль `unbound-redis` присутствует не во всех дистрибутивных сборках Unbound. Если модуль недоступен — вся архитектура Week 3 разрушается, блокируя P2 и задержку на 2-3 дня.

Митигация:

```
Уровень 1 (Основной):   Собрать Unbound из исходников с --enable-cachedb и
Redis-backend
                         (документировано в unbound.net, проверить на Week 1)
Уровень 2 (Fallback):   Написать DNS-прокси на Python (dnspython library):
                         Client → Python Proxy (Redis lookup) → Unbound (если
cache miss)
                         → Redis (store) → Client
Уровень 3 (Emergency):  Использовать PowerDNS Recursor + Lua scripting
(нативная поддержка
                         внешних backend через pdns-lmdb или custom Lua
hooks)
```

Действие: P3 проверяет наличие `--with-cachedb` в `unbound -V` на **День 2 (Week 1)**, не дожидаясь Week 3.

● Риск 2: Обход DNSSEC-валидации (Bottleneck Week 4)

Проблема: Если клиентская машина (не резолвер) самостоятельно проверяет DNSSEC (stub resolver с валидацией) — локальная подмена будет отклонена клиентом с `SERVFAIL`, и демонстрация bypass не состоится.

Митигация:

```
Действие 1: Использовать контролируемую клиентскую машину (VM без DNSSEC stub
validation),
            настроить /etc/resolv.conf → 10.10.0.2 (только наш резолвер)
Действие 2: Директива Unbound: domain-insecure: "nic.ru" + bogus-nxdomain
отключить
Действие 3 (Запасной домен): Зарегистрировать/использовать собственный домен
с DNSSEC,
            ТЗ явно это допускает ("можно реализовать на своём домене")
            → полный контроль над NS + DNSSEC-ключами = нет проблем с
валидацией
```

Действие: P1 тестирует `domain-insecure` директиву на **Week 2** (параллельно с 1.1), не ожидая Week 4.

● Риск 3: Воспроизводимость и полнота скриншотов (Bottleneck Week 4)

Проблема: Протокол со скриншотами (+15 баллов) часто собирается в последний момент. Если P4 не успевает — теряется 15% от максимального балла. Кроме того, скриншоты снятые "после" не подтверждают корректность эксперимента.

Митигация:

```
Правило 1: Ни один пункт ТЗ не считается DONE без скриншота в папке
/docs/screenshots/
Правило 2: P4 присутствует при выполнении КАЖДОГО пункта ТЗ в роли
наблюдателя-документатора
Правило 3: Использовать tmux + script(1) для автоматической записи
терминальных сессий
           (script -a /logs/week{N}_session.log) — страховка от "забытых"
скриншотов
Правило 4: Чеклист P4 блокирует Sprint Review — нельзя закрыть спринт без
протокола
```

Итоговая таблица баллов по дорожной карте

Неделя	Пункты ТЗ	Баллы	Статус
Week 2	1.1.А, 1.1.Б, 1.1.В, 1.1.Г	5+5+5+10 = **25**	Sprint 2
Week 3	1.2.А, 1.2.Б, 1.2.В, 1.2.Г	10+5+10+10 = **35**	Sprint 3
Week 4	1.3.А, 1.3.Б, 1.3.В, 1.3.Г	5+5+10+5 = **25**	Sprint 4
All weeks	Протокол со скриншотами	**+15**	P4 ongoing
ИТОГО		**100 баллов**	

💡 **Ключевой принцип:** Протокол — не послесловие, а параллельный артефакт. P4 документирует **в реальном времени**, иначе 15 бонусных баллов недостижимы.